

ĐỀ XUẤT DỰ ÁN HỆ THỐNG CƠ SỞ DỮ LIỆU PHÂN TÁN

DISTRIBUTED DATABASE PROJECT

PROPOSAL

Mẫu đề xuất Dự án môn học Cơ sở dữ liệu phân tán (Distributed Database Project Proposal)

Mã đề tài & Phân loại (Project ID & Category): Topic #112: Distributed Watermark Tracker (“Log Delay Compensator”) — Category 12

1

Project Identity (Thông tin Dự án)

- Sinh viên thực hiện (Student): Lê Đắc Quốc Anh
- Mã số sinh viên (MSSV): N23DCCN139
- Email: n23dccn139@student.ptithcm.edu.vn
- Project Title (Tên đề tài):
Hiện thực hóa Hệ thống định thời gian logic phân tán và so sánh hiệu quả giữa chiến lược định thời nghiêm ngặt (Strict Watermark) và định thời thích ứng (Heuristic Watermark) kết hợp Hàng đợi xử lý muộn (Dead-Letter Queue - DLQ).

2

Bảng thuật ngữ và Ký hiệu (Glossary)

Bảng dưới đây tổng hợp các ký hiệu toán học, biến trạng thái và tham số cấu hình được sử dụng xuyên suốt đề xuất, làm tham chiếu nhanh cho các công thức và sơ đồ phía sau:

Ký hiệu / Biến	Ý nghĩa	Giá trị / Phạm vi
T_{event}	Mốc thời gian logic (event-time) khi sự kiện được sinh ra.	giây
$T_{arrival}$	Mốc thời gian vật lý khi Worker tiếp nhận sự kiện.	giây
ℓ	Độ trễ truyền dẫn: $\ell = T_{arrival} - T_{event}$ (tương đương thời lượng chuyển đi đã nén).	giây
t_{ref}	Mốc tham chiếu nén thời gian (mốc khởi hành nhỏ nhất).	—
DIV	Hệ số nén thời gian (Time Compression).	60
$[window_start, win$	Khoảng Tumbling Window định danh một cửa sổ tổng hợp.	5 s

Ký hiệu / Biến	Ý nghĩa	Giá trị / Phạm vi
PartitionID	Phân mảnh ngang: $\text{hash}(\text{host}) \bmod 12$.	0–11
$LW_i(P_k)$	Watermark cục bộ của Worker i cho phân mảnh P_k (chế độ Strict).	giây
W_{global}	Watermark toàn cục Strict: $\min(LW_i)$ trên partition active.	giây
W_{heur}, W_h	Watermark thích ứng cục bộ (Heuristic).	giây
W_{global_h}	Watermark toàn cục Heuristic: $\min(W_h)$.	giây
L_{eff}	Độ trễ hiệu dụng: phân vị p của ℓ , tức $Q_p(\{\ell\})$.	giây
Q_p, p	Hàm phân vị (quantile) và phân vị mục tiêu.	$p \in [0.5, 0.999]$
α	Sai số tương đối (relative error) của DDSketch.	0.01
δ	Biên an toàn (Wait Time) cố định của chế độ Strict.	10 s
Ngưỡng nhàn rỗi	Ngưỡng thời gian đánh dấu phân mảnh nhàn rỗi tường minh (explicit idle).	30 s
Ngưỡng nhịp tim	Ngưỡng hết hạn nhịp tim để Coordinator xác nhận Worker lỗi.	10 s
$term$	Nhiệm kỳ (term) trong cơ chế đồng thuận, gắn vào Fencing Token chống split-brain.	số nguyên
window-id	Định danh cửa sổ duy nhất phục vụ phát kết quả idempotent.	—
Bảng 1: Bảng thuật ngữ, ký hiệu và tham số cấu hình của hệ thống Distributed Watermark Tracker.		

Lược đồ bản ghi sự kiện chuẩn hóa gồm năm trường: định danh máy chủ nguồn phát, mốc thời gian sự kiện logic, mốc thời gian tiếp nhận vật lý, dung lượng phản hồi và mã trạng thái giao dịch.

3 Objective & Problem Statement (Mục tiêu & Đặc tả bài toán)

- **The “Why” (Lý do thực hiện đề tài):**

Trong các hệ thống xử lý luồng dữ liệu phân tán (Distributed Stream Processing), việc đảm bảo tính đúng đắn khi thực hiện các phép tổng hợp theo cửa sổ thời gian (Windowed Aggregations) là một thách thức lớn do các sự kiện có thể đến muộn hoặc bị bất tuần tự (Out-of-order) bởi độ trễ truyền dẫn vật lý, sự cố mạng hoặc cơ chế đệm dữ liệu tại nguồn phát (Source Buffering).

Hệ thống cần xác định thời điểm thích hợp để đóng một cửa sổ và xuất kết quả. Điều này dẫn đến sự đánh đổi (trade-off) cốt lõi giữa **độ trễ chờ xử lý cửa sổ** (Watermark Latency) và **tính hoàn thiện dữ liệu** (Data Completeness). Đề tài này nhằm xây dựng một hệ thống phân tán để so sánh thực nghiệm hai phương thức định thời logic phổ biến:

1. *Định thời nghiêm ngặt (Strict Watermark)*: Cam kết **không mất dữ liệu (0% loss)** trên đường xử lý chính bằng cách chỉ tiến mốc thời gian toàn cục khi *mọi* phân mảnh đang hoạt động đã vượt qua mốc đó (xác nhận qua Punctuation Token từ nguồn). Theo phân loại của Özsu & Valduriez, đây là thiết kế **PC/EC** — ưu tiên tính nhất quán (Consistency) cả khi mạng phân tách lẫn lúc bình thường — nên chấp nhận chi phí điều phối cao và độ trễ tăng, đồng thời chịu ảnh hưởng bởi hiện tượng nút cổ chai do các phân mảnh chậm (Stragglers).
2. *Định thời thích ứng thống kê (Heuristic Watermark)*: Cam kết **độ trễ thấp**, trao quyền tự trị cục bộ tối đa cho Worker (*Site Autonomy*). Mỗi Worker tự ước lượng phân vị độ trễ thực nghiệm bằng cấu trúc DDSketch để đóng cửa sổ sớm mà không chờ phân mảnh khác, đồng thời định tuyến dữ liệu đến muộn sang Hàng đợi xử lý muộn (Dead-Letter Queue - DLQ) để đền bù bất đồng bộ. Đây là thiết kế **PA/EL** — ưu tiên tính sẵn sàng (Availability) và độ trễ (Latency) — chấp nhận nhất quán sau cùng (Eventual Consistency).

- **Core Logic (Thuật toán cốt lõi):**

- **Strict Path**: Trục thời gian logic (event-time) được cập nhật thông qua thông điệp kiểm soát *Punctuation Token* chứa mốc thời gian cam kết T_{commit} phát đi từ nguồn. Worker duy trì một hàng đợi ưu tiên Min-Heap cục bộ (Min-Heap) để sắp xếp lại dòng log. Coordinator tính toán mốc thời gian Watermark toàn cục theo thuật toán hội tụ tối thiểu:

$$W_{global}(t) = \min_{\forall P_k \in \text{Active}} LW_i(P_k) \quad (1)$$

- **Heuristic Path**: Sử dụng thuật toán *DDSketch* với thang chia logarit (Log-Scale Buckets) nhằm ước lượng phân vị p trực tuyến của độ trễ logic truyền dẫn $\ell = T_{arrival} - T_{event}$. Mốc thời gian Watermark thích ứng được tính theo công thức:

$$W_{heur}(i, p) = \max_{j < i} T_{event_max} - Q_p(\{\ell\}) \quad (2)$$

Các bản ghi trễ ($T_{event} < W_{global_heur}$) được chuyển hướng vào Hàng đợi xử lý muộn (Dead-Letter Queue — DLQ) để phục vụ quá trình đối chiếu đền bù trạng thái lịch sử (State Reconciliation).

4 Dataset Specification (Đặc tả Tập dữ liệu)

- **Source (Nguồn dữ liệu):**

Tập dữ liệu hành trình xe taxi màu vàng của Thành phố New York (NYC TLC Yellow Taxi Trip Records).

- **Size (Dung lượng tập dữ liệu):**

Tập dữ liệu chứa **2,961,423 hành trình** (sự kiện) được ghi nhận từ 260 máy chủ lưu trữ (hosts) giả lập, với dung lượng tệp CSV thô khoảng **159 MB**.

- **Cơ chế nén thời gian phù hợp với Engine (Time Compression Design):**

Do tập dữ liệu gốc trải dài trọn vẹn 31 ngày thực tế (khoảng 2,678,368 giây), việc chạy mô phỏng trực tiếp theo thời gian thực tế sẽ tốn rất nhiều thời gian và không khả thi cho môi trường kiểm thử. Vì vậy, hệ thống áp dụng kỹ thuật **Nén thời gian (Time Compression)** với hệ số nén $DIV = 60$.

Cơ chế nén hoạt động bằng cách chọn mốc thời gian bắt đầu hành trình nhỏ nhất làm tham chiếu $t_{ref} = \min(\text{pickup_timestamps})$, sau đó ánh xạ các thuộc tính thời gian như sau:

$$T_{event} = \frac{\text{pickup_timestamp} - t_{ref}}{60} \quad (3)$$

$$T_{arrival} = \frac{\text{dropoff_timestamp} - t_{ref}}{60} \quad (4)$$

Đặc tính quan trọng của phép nén:

Độ trễ truyền dẫn (lateness) logic của bản ghi được bảo toàn một cách tự nhiên theo thời gian chuyển đi được nén:

$$\ell = T_{arrival} - T_{event} = \frac{\text{dropoff_timestamp} - \text{pickup_timestamp}}{60} = \frac{\text{trip_duration}}{60} \quad (5)$$

Kỹ thuật này nén toàn bộ 31 ngày dữ liệu thực tế xuống còn **12.4 giờ** thời gian logic mô phỏng (giảm 60 lần), trong khi vẫn giữ nguyên các phân phối bất tuần tự đặc trưng (tỷ lệ giảm theo đúng hệ số $DIV = 60$):

- **Tỷ lệ bất tuần tự (out-of-order):** 49.41% sự kiện (1,463,199 bản ghi) đến lệch thứ tự so với sự kiện liền kề trước đó — đủ khốc liệt để kiểm thử cơ chế Watermark với cửa sổ 5 giây.
- Phân vị độ trễ mô phỏng: $p50 = 11.63$ s, $p90 = 28.80$ s, $p95 = 37.78$ s, $p99 = 59.70$ s; trễ cực đại (max lateness) = **359.73 s**.

Nếu phát lại dữ liệu theo tỷ lệ 1:1, độ trễ đến muộn gần như bằng 0 so với quãng 31 ngày, khiến đường cong hoàn thiện luôn phẳng ở 100% và không kiểm thử được Watermark. Cơ chế nén giữ nguyên hành vi bất tuần tự thực tế nhưng cho phép Worker thu thập chỉ số đánh đổi (completeness vs wait time) trong thời gian khả thi.

- **Schema (Cấu trúc dữ liệu):**

Hệ thống chuẩn hóa dữ liệu TLC Yellow Taxi thành mô hình sự kiện có cấu trúc:

Trường dữ liệu	Mô tả chi tiết
Máy chủ nguồn (host)	Định danh máy chủ nguồn phát (được ánh xạ từ thông tin vùng đón/trả).
Thời gian sự kiện (event-time)	Mốc thời gian logic sinh sự kiện (T_{event} , đơn vị: giây).
Thời gian tiếp nhận (arrival-time)	Mốc thời gian vật lý Worker nhận được sự kiện (T_{arrival} , đơn vị: giây).
Dung lượng phản hồi	Dung lượng dữ liệu (giả lập trường thông tin phản hồi của web server).
Mã trạng thái	Mã trạng thái giao dịch / phản hồi (ví dụ: 200, 404, 500).

- **Fragmentation Strategy (Chiến lược phân mảnh dọc/ngang):**

Áp dụng chiến lược phân mảnh ngang (Horizontal Fragmentation) dựa trên phương pháp băm (Hash Partitioning) trên thuộc tính máy chủ nguồn (host) để đảm bảo sự cân bằng tải giữa các node phân tán:

$$\boxed{PartitionID = \text{hash}(host) \bmod 12} \quad (6)$$

Tổng số 12 phân mảnh logic này được ánh xạ đều lên 4 Worker Nodes (như thể hiện trong sơ đồ miền xử lý dữ liệu):

- Worker 0 (node0) quản lý các phân mảnh: 0, 1, 2.
- Worker 1 (node1) quản lý các phân mảnh: 3, 4, 5.
- Worker 2 (node2) quản lý các phân mảnh: 6, 7, 8.
- Worker 3 (node3) quản lý các phân mảnh: 9, 10, 11.

5 System Architecture (Kiến trúc Hệ thống)

Hệ thống được triển khai dưới dạng đa container điều phối đồng bộ, mỗi thực thể tiến trình đảm nhận một vai trò xác định (Ingestor, Worker, Coordinator, Aggregator) và được cấu hình theo chế độ Strict hoặc Heuristic. Để làm rõ các luồng giao tiếp chuyên biệt và kiến trúc thành phần, hệ thống được phân rã thành các Domain chức năng và vai trò cụ thể như dưới đây.

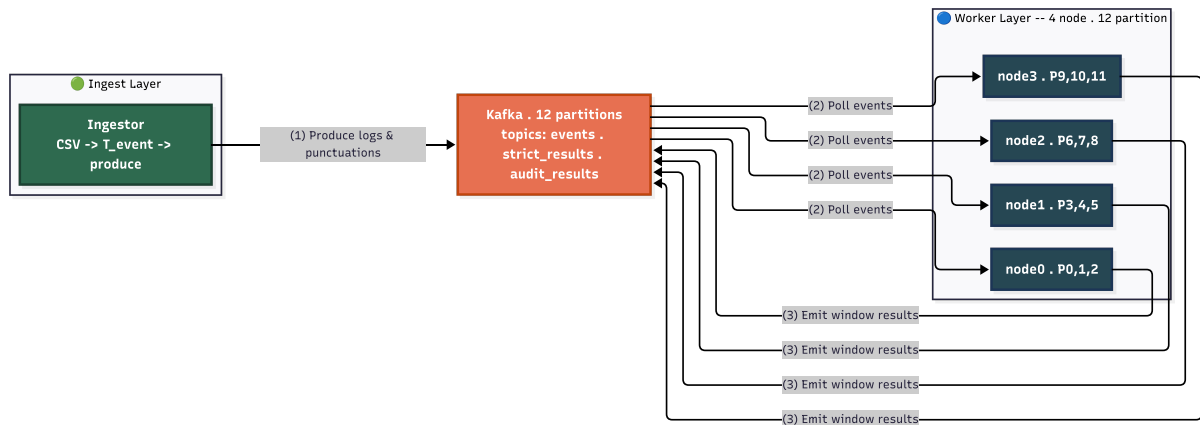
5.1 Phân rã kiến trúc theo các Domain thành phần

5.1.1 Domain 1: Luồng nạp dữ liệu từ Ingestor đến các Worker Nodes (Data Ingestion Domain)

Luồng nạp dữ liệu mô tả cách thức dữ liệu hành trình taxi được gắn thời gian sự kiện logic (*event-time*) tại nguồn và truyền dẫn phân tán thông qua hàng đợi Kafka để đưa về các Worker Nodes xử lý phân tán:

- **Ingestor Layer:** Tiến trình Ingestor thực hiện đọc tập dữ liệu hành trình xe taxi từ tệp CSV, tính toán gắn nhãn thời gian sự kiện logic T_{event} , sau đó ghi các bản ghi sự kiện kèm theo các token kiểm soát (*Punctuation Tokens* trong chế độ Strict) vào dòng sự kiện trên Kafka.
- **Data Plane (Kafka Broker):** Tổ chức lưu trữ tạm thời dòng sự kiện bất tuần tự trên 12 phân mảnh logic (partitions) của dòng sự kiện.
- **Worker Layer:** 4 thực thể Worker Nodes chạy song song thực hiện kéo (poll) dữ liệu liên tục từ các phân mảnh Kafka được chỉ định về sắp xếp lại tại Min-Heap cục bộ trước khi tính toán.
- **Output Layer:** Khi các cửa sổ thời gian được chốt, Worker phát kết quả cửa sổ vào dòng kết quả chính và nhân bản song song sang một dòng kiểm toán chuyên biệt (Critical Audit Sink).

Sơ đồ mô tả luồng nạp dữ liệu:



Hình 1: Sơ đồ Domain 1: Luồng nạp dữ liệu từ Ingestor đến các Worker Nodes (Data Ingestion Domain).

Luồng giao tiếp	Hướng	Giao thức / Cổng	Nội dung & Tần suất
Phát log punctuation	+ Ingestor → dòng sự kiện Kafka	Kafka Producer (xác nhận đầy đủ)	Bản ghi sự kiện (đã gán T_{event}) kèm Punctuation Token (T_{commit}); phân mảnh theo hàm băm máy chủ nguồn. Khi phân mảnh nhàn rỗi phát Empty Punctuation mỗi khoảng 1s (chỉ Strict).
Poll sự kiện	dòng sự kiện Kafka → Worker (N0–N3)	Kafka Consumer (pull)	Mỗi Worker poll phân mảnh được gán, nạp Bounded Min-Heap (sắp theo thời gian sự kiện); cơ chế backpressure tạm dừng/tiếp tục consume chống tràn RAM.
Phát kết quả cửa sổ	Worker → dòng kết quả Kafka	Kafka Producer (xác nhận đầy đủ)	Phát kết quả khi biên phải cửa sổ $\leq W$; idempotent theo định danh cửa sổ; nhân bản song song sang Critical Audit Sink.

Bảng 2: Chi tiết các luồng giao tiếp trong Domain 1 (Data Ingestion).

5.1.2 Domain 2: Luồng điều phối Strict Watermark (Strict Coordination Domain)

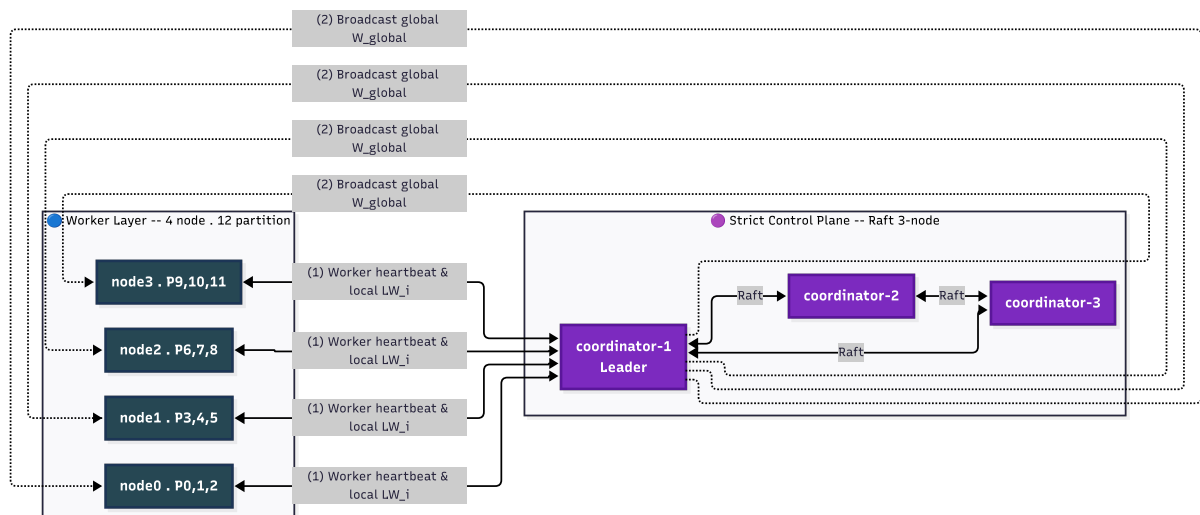
Để cam kết bảo toàn dữ liệu tuyệt đối (0% mất mát dữ liệu), luồng Strict Watermark duy trì cơ chế hội tụ toàn cục đồng thuận:

- **Local Watermark:** Các Worker Node nhận dữ liệu, tính toán Watermark cục bộ $LW_i(P_k)$ cho từng phân mảnh đang quản lý và gửi báo cáo thông qua luồng nhịp tim (heartbeat) về Coordinator.
- **High-Availability Coordinator Plane:** Cụm điều phối gồm 3 instance hoạt động theo thuật toán đồng thuận Raft (Leader-Follower). Coordinator Leader định kỳ tiếp nhận nhịp tim từ toàn bộ các Worker, tính toán Watermark toàn cục bằng giá trị nhỏ nhất của tất cả các phân mảnh hoạt động:

$$W_{global}(t) = \min_{\forall P_k \in \text{Active}} LW_i(P_k) \quad (7)$$

Mức Watermark toàn cục W_{global} sau đó được broadcast ngược lại cho các Worker Nodes làm căn cứ chốt và giải phóng trạng thái cửa sổ.

Sơ đồ mô tả điều phối Strict:



Hình 2: Sơ đồ Domain 2: Luồng điều phối Strict Watermark (Strict Coordination Domain).

Luồng giao tiếp	Hướng	Giao thức / Cổng	Nội dung & Tần suất
Worker Heartbeat	Worker Coordinator Leader	→ gRPC (kênh HTTP dự phòng)	Bản đồ phân mảnh $\rightarrow LW_i$, kèm event-time lớn nhất, offset đã xử lý, Fencing Token và danh sách phân mảnh nhàn rỗi; chu kỳ 1s.
Truy vấn trạng thái toàn cục	Worker Coordinator Leader	→ gRPC (kênh HTTP dự phòng)	Trả W_{global} , nhiệm kỳ hiện tại và phân loại trạng thái phân mảnh; Worker pull mỗi 500ms.
Đồng thuận & Bầu chọn HA	Leader Followers ZooKeeper	$\leftrightarrow$ Đồng thuận Raft hoặc khóa ZooKeeper	Chế độ Raft: sao chép W_{global} , sơ đồ phân bổ phân mảnh và nhiệm kỳ; commit khi đa số (2/3) xác nhận. Chế độ ZooKeeper: Leader sở hữu khóa phân tán và đồng bộ trạng thái tới các follower.
Failover Failback	/ Coordinator Worker	→ gRPC/HTTP kèm cặp (nhiệm kỳ, định danh lệnh)	Lệnh tái phân bổ / tạm dừng / tiếp tục có Fencing Token; từ chối lệnh có nhiệm kỳ cũ.
Broadcast W_{global}	Coordinator (nội bộ)	Vòng lặp chủ động 200ms	Tính lại và đẩy W_{global} kèm trạng thái phân mảnh phục vụ đồng bộ và đo lường.

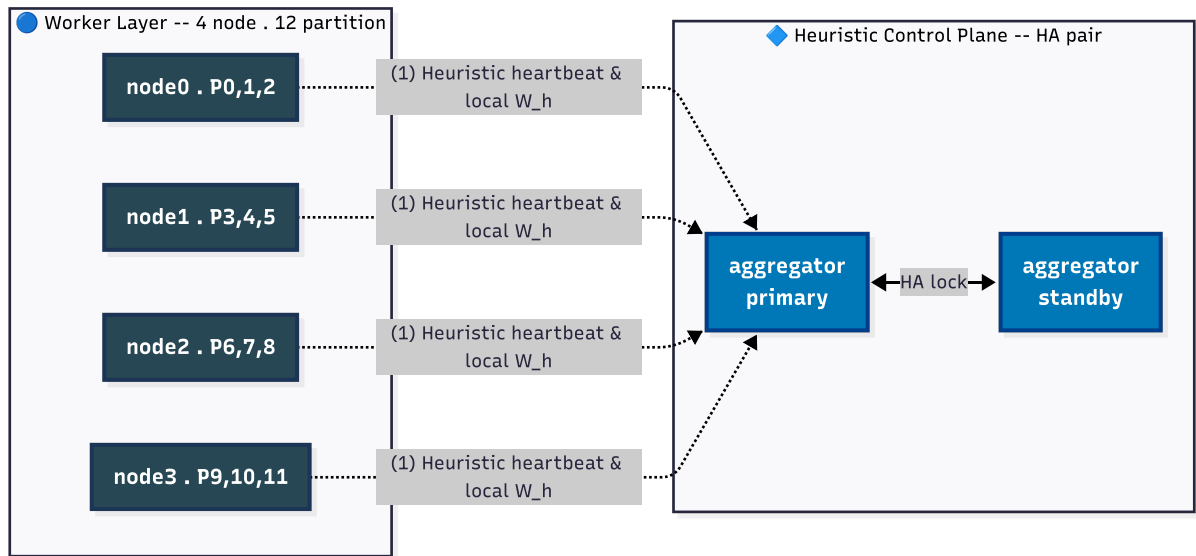
Bảng 3: Chi tiết các luồng giao tiếp trong Domain 2 (Strict Coordination).

5.1.3 Domain 3: Luồng điều phối Heuristic Watermark (Heuristic Aggregation Domain)

Đối với chế độ tối ưu hóa độ trễ, luồng Heuristic Watermark sử dụng cấu trúc gom hợp thống kê bất đồng bộ:

- **Statistical Heartbeat:** Các Worker sử dụng cấu trúc dữ liệu DDSketch cục bộ để đo đặc độ trễ thực tế, báo cáo mốc Watermark thích ứng W_h và bản chụp (snapshot) DDSketch cục bộ tới Aggregator.
- **Active-Standby Control Plane:** Cặp Aggregator dự phòng nóng (Primary-Standby). Việc tranh chấp Leader được thực hiện qua khóa phân tán ZooKeeper hoặc khóa tệp dùng chung (hỗ trợ đa nền tảng). Standby liên tục theo dõi nhịp tim của Active (ghi mỗi 1s). Nếu nhịp tim mất hoặc quá hạn ($> 1.5s$), Standby tự động chiếm khóa và được nâng lên làm Active, tải lại trạng thái phân mảnh và watermark đã lưu bền vững. Aggregator Active tổng hợp thông tin thu nhận để điều phối watermark thích ứng cục bộ, bản ghi mượn đi vào Hàng đợi xử lý mượn (Dead-Letter Queue — DLQ).

Sơ đồ mô tả điều phối Heuristic:



Hình 3: Sơ đồ Domain 3: Luồng điều phối Heuristic Watermark (Heuristic Aggregation Domain).

Luồng tiếp	giao	Hướng	Giao thức / Cổng	Nội dung & Tần suất
Heuristic Heartbeat		Worker Aggregator Primary	→ gRPC (kênh HTTP dự phòng)	Định danh Worker, định danh phân mảnh, W_h kèm L_{eff} và snapshot DDSketch; chu kỳ khoảng 200ms.
Tính watermark toàn cục		Aggregator (nội bộ)	Vòng lặp 500ms	$W_{global_h} = \min(W_h)$ trên phân mảnh hoạt động; phân loại trạng thái phân mảnh (hoạt động / chậm / nhàn rỗi / lỗi).
HA Standby	Active-	Primary Standby	↔ Khóa ZooKeeper / khóa tệp + trạng thái dùng chung	Standby theo dõi nhịp tim ghi mỗi 1s. Mất nhịp tim $> 1.5s \Rightarrow$ Standby chiếm khóa, khôi phục trạng thái bền vững và lên Active ($RTO \leq 2s$).
Broadcast W_{global_h}		Aggregator Worker	→ HTTP (pull)	Worker nhận W_{global_h} thích ứng để chốt cửa sổ.
Định tuyến dữ liệu muộn		Worker trên Kafka	→ DLQ Kafka Producer	Sự kiện $T_{event} < W_h$ được đẩy vào DLQ; hạch toán hao hụt theo cửa sổ thay vì chặn dòng chính.

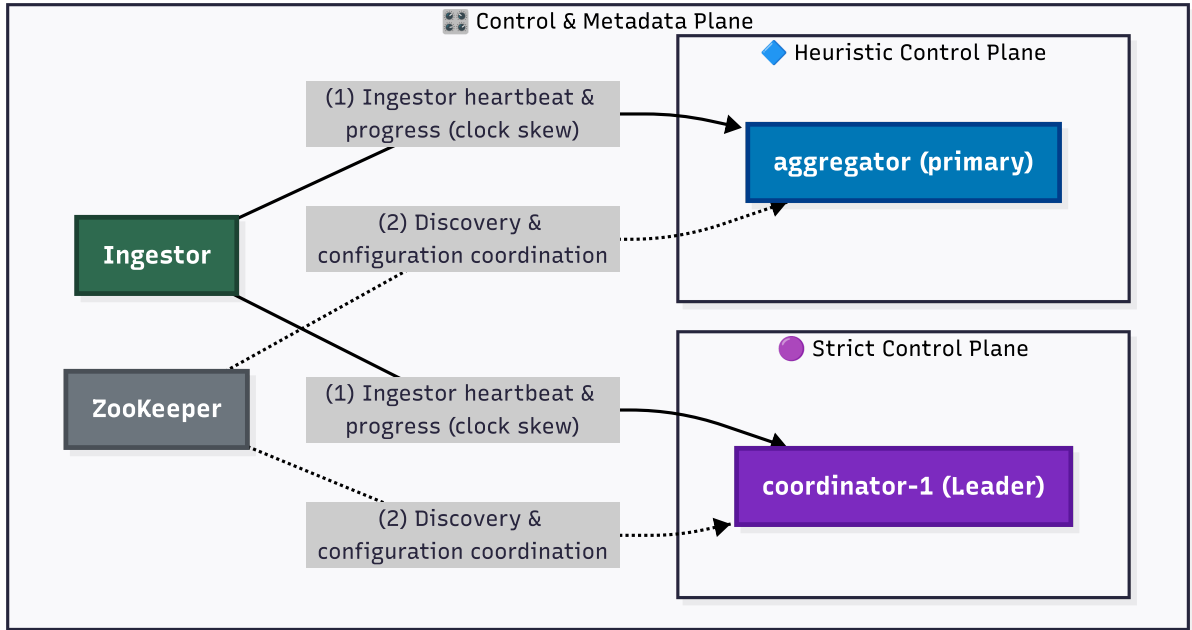
Bảng 4: Chi tiết các luồng giao tiếp trong Domain 3 (Heuristic Aggregation).

5.1.4 Domain 4: Phối hợp Control & Metadata Plane (Control & Infrastructure Domain)

Mặt phẳng điều khiển và quản lý cấu hình hệ thống:

- **Ingestor Heartbeat:** Ingestor định kỳ gửi thông tin tiến độ đọc dữ liệu và độ lệch đồng hồ vật lý (clock skew) để Coordinator giám sát sức khỏe và độ lệch đồng hồ. Để tránh hiện tượng nghẽn hội tụ gRPC (fan-in bottleneck), Ingestor đẩy heartbeat trực tiếp qua một dòng nhịp tim trên Kafka, sử dụng gRPC/HTTP làm kênh dự phòng.
- **Metadata Plane (ZooKeeper):** ZooKeeper đóng vai trò trung tâm lưu trữ thông tin cấu hình cluster Kafka, thực hiện Service Discovery và hỗ trợ bầu chọn Leader cho các thành phần điều phối phân tán.

Sơ đồ mô tả Control & Metadata Plane:



Hình 4: Sơ đồ Domain 4: Phối hợp Control & Metadata Plane (Control & Infrastructure Domain).

Luồng tiếp	giao	Hướng	Giao thức / Cổng	Nội dung & Tần suất
Ingestor Heartbeat	Ingestor → Kafka / Coordinator		Dòng nhịp tim Kafka hoặc gRPC dự phòng	Định danh Ingestor, mốc cam kết T_{commit} , timestamp, danh sách phân mảnh được gán, đồng hồ nguồn và offset; gửi mỗi 5s; Coordinator đo RTT để giám sát sức khỏe và độ lệch đồng hồ.
Truy vấn giám sát	Dashboard / Ops → Coordinator & Worker		HTTP (giao diện giám sát)	Bảng sức khỏe / độ lệch đồng hồ, chỉ số completeness/latency, trạng thái W_{global} , nhiệm kỳ và failover.
Discovery & cấu hình	ZooKeeper ↔ Kafka / Coordinator / Aggregator		ZooKeeper	Lưu cấu hình cụm Kafka, bầu chọn Leader và hỗ trợ Service Discovery.

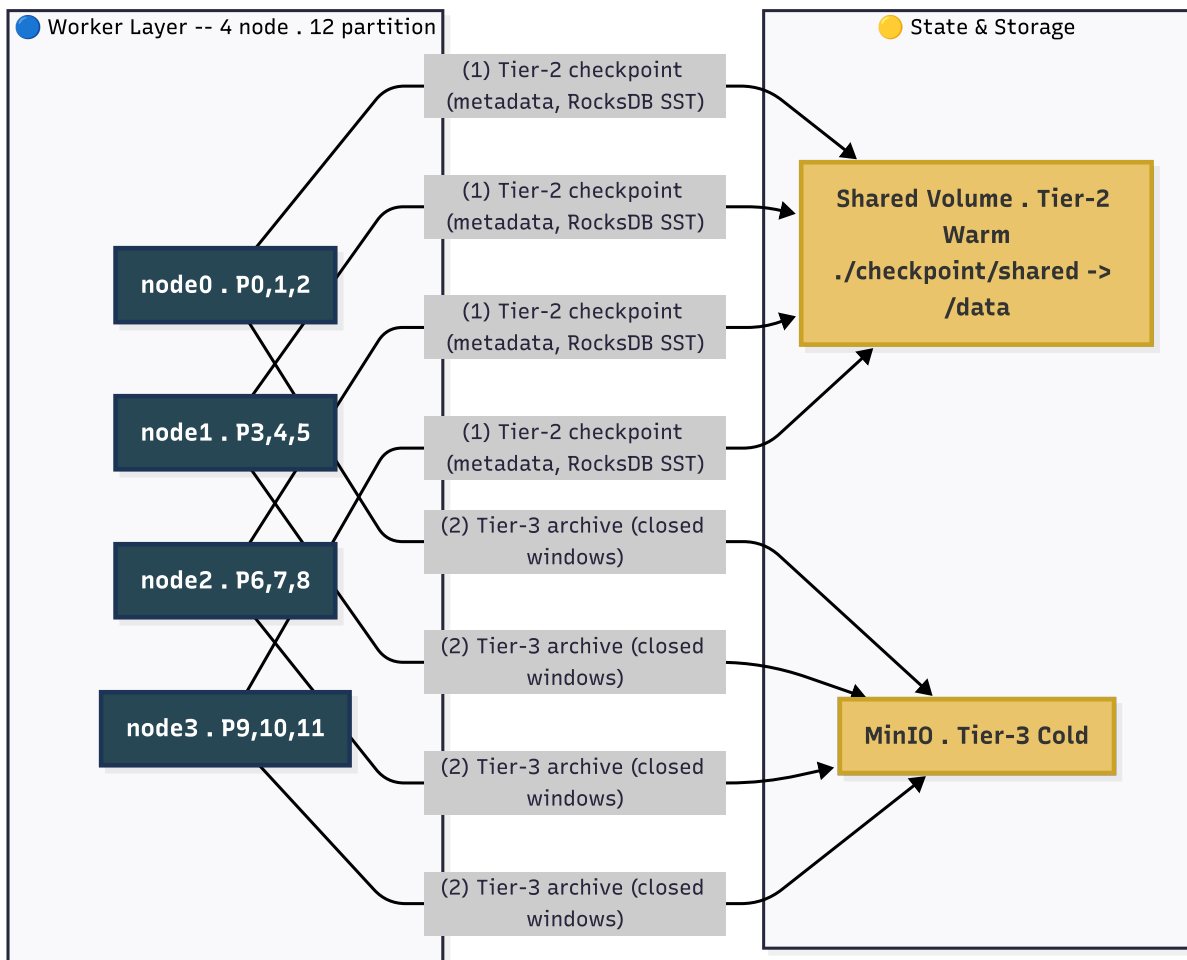
Bảng 5: Chi tiết các luồng giao tiếp trong Domain 4 (Control & Metadata).

5.1.5 Domain 5: Luồng lưu trữ trạng thái và Checkpoint (State & Storage Domain)

Cơ chế quản lý lưu trữ trạng thái phân tầng (Tiered State Storage) phục vụ mục tiêu khôi phục sau sự cố:

- **Tier-1 Hot State:** Mỗi Worker ghi nhận trạng thái của sổ đang mở trực tiếp vào RocksDB nhúng cục bộ trên SSD NVMe để đảm bảo độ trễ < 1 ms.
- **Tier-2 Warm State:** Định kỳ 10 giây, Worker sao lưu trạng thái xử lý logic và tệp dữ liệu RocksDB cục bộ sang Shared Volume chung phục vụ việc tiếp quản tức thời khi có node gặp sự cố.
- **Tier-3 Cold State:** Các kết quả của sổ đã chốt và dữ liệu lịch sử hoàn chỉnh được lưu trữ nén lâu dài trên Object Storage tương thích S3 (MinIO).

Sơ đồ mô tả lưu trữ và Checkpoint:



Hình 5: Sơ đồ Domain 5: Luồng lưu trữ trạng thái và Checkpoint (State & Storage Domain).

Luồng tiếp	giao	Hướng	Giao thức / Cổng	Nội dung & Tần suất
Tier-1 State	Hot	Worker RocksDB cục bộ	$\leftrightarrow$ RocksDB nhúng (cô lập theo phân mảnh)	Đọc/ghi trạng thái cửa sổ < 1ms; phân vùng không gian khóa cho cửa sổ đang mở, cửa sổ đã chốt, bảng khử trùng lặp và siêu dữ liệu.
Tier-2 Checkpoint		Worker Shared Volume	$\rightarrow$ Ghi tệp định kỳ	Mỗi 10s ghi trạng thái xử lý, tập tin RocksDB và offset Kafka đã commit; phục vụ failover (node khác tiếp quản và tiếp tục từ offset kế tiếp).
Tier-3 Archive / DR		Worker MinIO (S3)	$\rightarrow$ MinIO client	Tải lên cửa sổ đã chốt theo khóa tắt định (phân mảnh / định danh cửa sổ); vòng đời loại bỏ trạng thái: đã chốt $\rightarrow$ đang tải lên $\rightarrow$ đã lưu $\rightarrow$ đã dọn; sao lưu định kỳ phục vụ Disaster Recovery.

Bảng 6: Chi tiết các luồng giao tiếp trong Domain 5 (State & Storage).

5.2 Vai trò của các thành phần trong hệ thống (Container View)

Dưới đây là chi tiết vai trò của các thực thể trong kiến trúc hệ thống xử lý luồng Watermark phân tán:

Ký hiệu Node	Phân lớp (Layer)	Mô tả vai trò & Chức năng chính
ING	Ingestor	Đọc dữ liệu CSV từ thư mục chia sẻ, gán nhãn thời gian sự kiện (T_{event}), gửi dữ liệu và Punctuation Token vào Kafka.
KAFKA	Data Plane	Broker điều phối thông điệp, lưu trữ 12 phân mảnh cho các dòng sự kiện đầu vào, dòng kết quả chính và dòng kiểm toán.
ZK	Auxiliary	Quản lý cấu hình cluster Kafka, hỗ trợ bầu chọn Leader (Coordinator/Aggregator) và phát hiện dịch vụ (Service Discovery).
N0 – N3	Worker Layer	Các thực thể tính toán chạy song song, tiêu thụ dữ liệu từ các partition Kafka được phân bổ, quản lý window và tính toán Watermark cục bộ.
C1 – C3	Strict Plane	Control Cụm điều phối trạng thái hoạt động theo giải thuật đồng thuận Raft (Leader-Follower) để tính toán Watermark toàn cục (W_{global}) trong chế độ Strict.

Ký hiệu Node	Phân lớp (Layer)	Mô tả vai trò & Chức năng chính
AGG / AGGS	Heuristic Control Plane	Cấp cấu hình dự phòng nóng (HA qua khóa phân tán) để thu thập dữ liệu thống kê từ Workers và điều phối Watermark toàn cục trong chế độ Heuristic.
VOL	Storage (Tier-2)	Phân vùng lưu trữ chia sẻ cục bộ, lưu checkpoint trạng thái (mỗi 10s) của các worker để hỗ trợ Failover tức thời.
MINIO	Storage (Tier-3)	Kho lưu trữ đối tượng dạng S3 tương thích, lưu trữ lâu dài các kết quả của các window đã được chốt (Purged/Closed).
DASH	Dashboard sát	Giáo diện đồ họa theo dõi hiệu năng hệ thống (completeness vs wait time) và điều khiển vòng đời các container.

Bảng 7: Bảng đặc tả chi tiết vai trò các thành phần cấu trúc hệ thống.

5.3 Vai trò của các tiến trình thực thi (Executable Roles)

Mỗi tiến trình của hệ thống được khởi chạy dưới một trong bốn vai trò xác định, được lựa chọn qua tham số cấu hình khi triển khai:

Vai trò (Role)	Mô tả chức năng
Ingestor	Đọc dữ liệu nguồn, gán mốc thời gian sự kiện T_{event} , phát bản ghi sự kiện kèm Punctuation Token vào dòng sự kiện Kafka.
Worker	Poll dữ liệu từ Kafka, sắp xếp lại theo event-time bằng hàng đợi ưu tiên có giới hạn, khử trùng lặp, gom cửa sổ, lưu trạng thái RocksDB và phát kết quả.
Coordinator	Cụm 3 node đồng thuận Raft, tính $W_{global} = \min(LW_i)$, điều phối failover và kiểm soát lệnh bằng Fencing Token.
Aggregator	Cặp Active-Standby (HA qua khóa phân tán), tổng hợp W_{global_h} thích ứng từ các snapshot DDSketch của Worker.

Bảng 8: Bảng đặc tả các tiến trình thực thi của hệ thống.

5.4 Hợp đồng giao tiếp giữa các Domain và Giải pháp cho bài toán nút cổ chai (Stragglers)

Để hệ thống hoạt động đồng bộ và bền vững, các giao tiếp giữa các Domain được chuẩn hóa thành các hợp đồng dịch vụ (Service Contracts) chặt chẽ:

1. **Hợp đồng Ingestion (Data Ingestion Contract):** Ingestor cam kết phân mảnh dữ liệu hành trình xe taxi ngang theo hàm băm và gán thời gian sự kiện logic

T_{event} . Trong chế độ nghiêm ngặt (Strict), Ingestor phải phát thông điệp kiểm soát (Punctuation Token) chứa mốc cam kết T_{commit} . Khi một phân mảnh không có chuyển đi mới phát sinh (nhàn rỗi), Ingestor định kỳ gửi các *Empty Punctuation Token* với T_{commit} tăng dần để dòng chảy thời gian của phân mảnh đó không bị dừng.

2. **Hợp đồng Heartbeat của Worker (Worker Heartbeat Contract):** Định kỳ 1 giây, các Worker gửi báo cáo trạng thái qua gRPC tới Coordinator. Nội dung báo cáo bao gồm: mốc thời gian Watermark cục bộ của từng phân mảnh đang quản lý $LW_i(P_k)$, offset Kafka đã xử lý, và danh sách các phân mảnh bị nhàn rỗi.
3. **Hợp đồng Trạng thái Toàn cục (Global State Contract):** Các Worker định kỳ 500 ms truy vấn Coordinator để nhận mốc thời gian Watermark toàn cục W_{global} phục vụ cho việc chốt và giải phóng trạng thái của sổ thời gian logic.

5.4.1 Cơ chế giải quyết bài toán nút cổ chai (Stragglers)

Hiện tượng nút cổ chai xảy ra khi một hoặc một vài phân mảnh bị xử lý chậm trễ hoặc không có dữ liệu phát sinh (Idle Partition), kéo mốc Watermark toàn cục dừng lại và làm tràn bộ nhớ đệm trạng thái RAM (MemTable) tại tất cả các Worker khác. Hệ thống giải quyết bài toán này qua hai chiến lược chuyên biệt:

Giải pháp trong chế độ Strict (Empty Punctuation & Idleness Bypass):

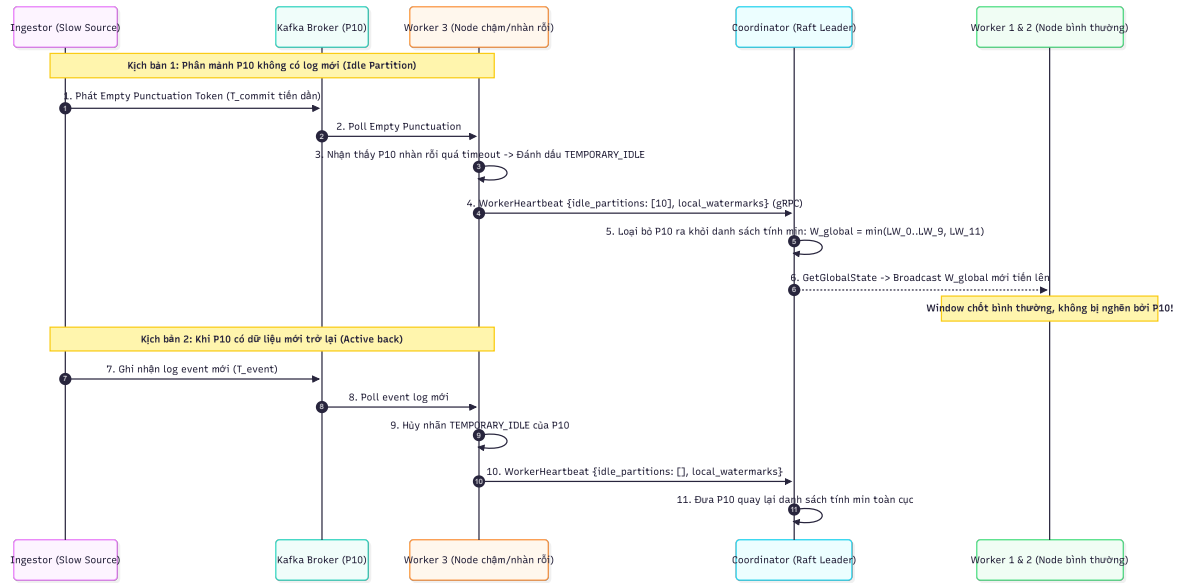
- Khi Worker nhận được Empty Punctuation Token từ Kafka, nó sẽ tịnh tiến watermark cục bộ $LW_i(P_k)$ của phân mảnh đó lên.
- Nếu một phân mảnh nhàn rỗi quá ngưỡng thời gian quy định (mặc định 30s), Worker sẽ tự động đánh dấu phân mảnh đó là nhàn rỗi tường minh (explicit idle) và gửi báo cáo danh sách này lên Coordinator.
- Coordinator Leader khi tính toán watermark toàn cục theo công thức hội tụ tối thiểu *đơn điệu*:

$$W_{global}(t) = \max \left(W_{global}^{prev}, \min_{P_k \notin Idle} LW_i(P_k) \right) \quad (8)$$

Sẽ tạm thời loại trừ các phân mảnh nhàn rỗi tường minh ra khỏi hàm min, cho phép W_{global} tiếp tục tiến lên, giải phóng trạng thái của sổ của các phân mảnh hoạt động khác. Khi phân mảnh nhàn rỗi có dữ liệu trở lại, nó sẽ tự động được đưa trở lại danh sách tính toán.

- **Bất biến bảo toàn dữ liệu:** Empty Punctuation là cơ chế *chính* giữ W_{global} tịnh tiến nên hệ thống **không** loại bỏ ngầm các phân mảnh chỉ vì im lặng. Các phân mảnh phản hồi chậm (STALE) và nhàn rỗi ngầm (IDLE) vẫn nằm trong hàm min(); chỉ phân mảnh được xác nhận lỗi (FAILED) hoặc được Worker đánh dấu nhàn rỗi tường minh mới bị loại trừ tạm thời, bảo toàn cam kết 0% loss.

Sơ đồ trình tự xử lý phân mảnh nhàn rỗi bằng Empty Punctuation trong chế độ Strict:

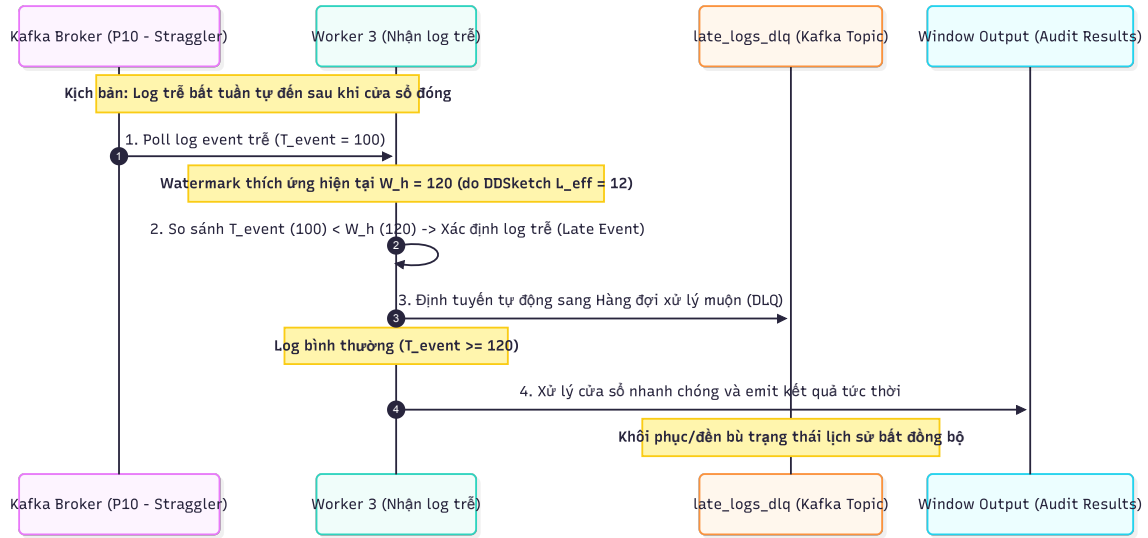


Hình 6: Trình tự xử lý Straggler chế độ Strict: Empty Punctuation & Idleness Bypass.

Giải pháp trong chế độ Heuristic (DDSketch & DLQ routing):

- Không có sự phụ thuộc toàn cục. Mỗi Worker tự ước lượng phân phối độ trễ cục bộ của phân mảnh đang giữ bằng cấu trúc dữ liệu DDSketch và tự xác định mốc chốt cửa sổ thích ứng W_h mà không cần chờ đợi các phân mảnh khác.
- Nếu một phân mảnh bị chậm (Straggler), các sự kiện đến muộn sau khi cửa sổ đã chốt sẽ không làm tắc nghẽn luồng xử lý chính. Chúng sẽ được định tuyến tự động vào Hàng đợi xử lý muộn (Dead-Letter Queue - DLQ) để xử lý đền bù trạng thái lịch sử sau đó, đảm bảo độ trễ chốt cửa sổ luôn ở mức tối thiểu.

Sơ đồ trình tự định tuyến dữ liệu muộn sang DLQ trong chế độ Heuristic:



Hình 7: Trình tự xử lý Straggler chế độ Heuristic: DDSketch ước lượng & định tuyến DLQ.

5.5 Đặc tả Giao tiếp Chi tiết (gRPC Contracts & HTTP REST APIs)

Hệ thống thiết lập một cơ chế giao tiếp đồng bộ và bất đồng bộ hai tầng vững chắc bao gồm: (1) các giao dịch gRPC hiệu năng cao phục vụ trao đổi siêu dữ liệu, đồng thuận Raft và nhịp tim trạng thái; và (2) giao diện REST HTTP dùng để kiểm tra sức khỏe (health checks), thu thập chỉ số Prometheus, và cung cấp kênh giao tiếp dự phòng (fallback paths).

5.5.1 Giao diện gRPC (Mặt phẳng điều khiển)

Mặt phẳng điều khiển trao đổi siêu dữ liệu qua hai dịch vụ gRPC hiệu năng cao:

1. **Dịch vụ Coordinator (Strict Control Plane):** Chịu trách nhiệm tiếp nhận báo cáo tiến trình từ Worker và Ingestor, điều phối phân mảnh, đồng bộ trạng thái đồng thuận và bầu cử Leader. Các tương tác cốt lõi gồm:
 - **Worker Heartbeat:** Worker gửi nhịp tim định kỳ (1s), báo cáo mốc thời gian logic của từng phân mảnh để Coordinator tính W_{global} .
 - **Worker Liveness Ping:** Worker kiểm tra liveness và đăng ký danh mục phân mảnh với Coordinator.
 - **Ingestor Heartbeat:** Ingestor gửi tiến độ nạp dữ liệu và mốc cam kết T_{commit} của nguồn.
 - **Truy vấn Trạng thái Toàn cục:** Worker truy vấn định kỳ (500ms) để lấy mốc W_{global} mới nhất.
 - **Bầu chọn & Nhân bản đồng thuận:** phục vụ bầu Leader và nhân bản nhật ký giữa 3 thực thể Coordinator HA (qua đồng thuận Raft hoặc khóa ZooKeeper dự phòng).

2. **Dịch vụ Aggregator (Heuristic Control Plane):** tiếp nhận mốc Watermark thích ứng W_h cục bộ của từng phân mảnh từ các Worker tự trị để tổng hợp W_{global_h} một cách bất đồng bộ.

Bảng 9 mô tả nội dung các thông điệp trao đổi chính theo chức năng:

Thông điệp	Nội dung chức năng
Nhịp tim Worker	Định danh Worker; bản đồ phân mảnh $P_k \rightarrow LW_i(P_k)$; mốc event-time lớn nhất đã đọc; nhiệm kỳ điều phối hiện tại (Fencing Token); danh sách phân mảnh tạm thời nhàn rỗi.
Nhịp tim Ingestor	Định danh Ingestor; mốc cam kết thời gian của nguồn T_{commit} ; timestamp vật lý lúc gửi; danh sách phân mảnh được gán.
Phản hồi trạng thái toàn cục	Watermark toàn cục W_{global} hiện tại; nhiệm kỳ Coordinator đang hoạt động; sơ đồ phân loại trạng thái phân mảnh.
Báo cáo Watermark thích ứng	Định danh Worker; định danh phân mảnh; mốc Watermark thích ứng W_h cục bộ gửi lên Aggregator.

Bảng 9: Nội dung chức năng các thông điệp điều khiển chính trong hệ thống.

5.5.2 Giao diện HTTP REST (Giám sát & Điều khiển)

Mỗi thực thể (Worker, Coordinator, Aggregator) cung cấp một giao diện HTTP nội bộ phục vụ giám sát, thu thập chỉ số và một số kênh điều khiển dự phòng, được phân nhóm theo chức năng:

- **Giám sát sức khỏe và chỉ số:** truy vấn trạng thái hoạt động và vai trò của tiến trình; tín hiệu sẵn sàng tiếp nhận dữ liệu (phân biệt pha khởi động/khôi phục); chỉ số hiệu năng theo định dạng chuẩn quan trắc; và báo cáo chi tiết chỉ số nội bộ của Worker (mức sử dụng RAM/CPU, kích thước hàng đợi Min-Heap, số lần kích hoạt backpressure, dung lượng trạng thái RocksDB).
- **Truy vấn điều khiển và siêu dữ liệu:** trạng thái logic toàn cục (W_{global} , nhiệm kỳ hiện tại, trạng thái các phân mảnh được phân bổ); thông tin bộ quản lý failover (bảng phân bổ phân mảnh và sức khỏe từng Worker); trạng thái nghẽn dòng; và thông tin sức khỏe Ingestor (tiến độ đọc T_{commit} , độ lệch đồng hồ vật lý và độ trễ mạng vòng).
- **Kênh nạp dữ liệu và nhịp tim dự phòng:** tiếp nhận trực tiếp sự kiện log, thông điệp Punctuation từ Ingestor (chế độ Strict), báo cáo nhịp tim Ingestor, và chỉ thị tái phân bổ phân mảnh khi kênh gRPC chính bị gián đoạn.

6 Tech Stack & Implementation Plan (Công nghệ & Kế hoạch triển khai)

- **Programming Language (Ngôn ngữ lập trình):** Python 3.10+ (Đảm bảo hiệu năng lập trình nhanh, thư viện phân tích dữ liệu phong phú).
- **Deployment (Phương thức triển khai):** Đóng gói bằng *Docker* và điều phối đa container qua *Docker Compose*. Cấu hình hai hồ sơ (profiles) triển khai độc lập để khởi động nhanh chóng hệ thống ở chế độ Strict hoặc Heuristic chỉ với một lệnh duy nhất:
 - Strict Profile: Khởi chạy Coordinator và 3 Worker chế độ nghiêm ngặt.
 - Heuristic Profile: Khởi chạy Aggregator và các Worker chế độ thích ứng trễ.
- **Libraries/Frameworks (Các thư viện cốt lõi):**
 - *grpcio* & *grpcio-tools*: Xây dựng giao diện RPC hiệu năng cao.
 - *pyrocksdb* hoặc *RocksDB* binding: Quản lý lưu trữ trạng thái tại Worker.
 - *DDSketch* implementation: Thuật toán phân vị thích ứng với sai số giới hạn $\alpha = 0.01$.
 - *Pandas*, *Numpy*, *Matplotlib*: Thống kê dữ liệu thực nghiệm, tính toán bootstrap CI và vẽ đồ thị Pareto.
 - *pytest*: Framework viết kiểm thử tự động (unit test & integration test).

7 Success Metrics & Analysis (Chỉ số đánh giá & Phân tích kịch bản sự cố)

7.1 Completeness-at-Latency Trade-off (Sự đánh đổi giữa tính hoàn thiện dữ liệu và độ trễ)

Để đánh giá hiệu quả của hai phương thức định thời logic, hệ thống thực hiện đo lường hai chỉ số định lượng cốt lõi:

1. Tính hoàn thiện dữ liệu (Data Completeness %):

- *Độ hoàn thiện tức thời (Immediate Completeness)*: Tỷ lệ dữ liệu xử lý đúng hạn trong cửa sổ logic trước khi mốc thời gian Watermark đóng cửa sổ đó.
- *Độ hoàn thiện cuối cùng (Eventual Completeness)*: Tỷ lệ dữ liệu được tổng hợp đầy đủ sau khi đã tính cả các bản ghi đến muộn được xử lý đền bù qua Hàng đợi xử lý muộn (Dead-Letter Queue - DLQ).

2. Độ trễ chờ Watermark (Watermark Latency / Wait Time):

Khoảng thời gian hệ thống phải chờ (tính bằng giây mô phỏng) từ lúc cửa sổ thời gian logic kết thúc cho đến khi mốc thời gian Watermark toàn cục vượt qua biên đóng cửa sổ.

Chúng tôi đã thực hiện chạy quét thực nghiệm (benchmark sweeps) trên toàn bộ tập dữ liệu **2,961,423 bản ghi**, áp dụng cơ chế nén thời gian $DIV = 60$ và phân phối độ trễ đã mô tả ở Mục 4 (Dataset Specification). Dưới đây là các phân tích chuyên sâu về sự đánh đổi này.

7.1.1 Kết quả quét thực nghiệm so sánh trực tiếp

Kết quả so sánh trực tiếp giữa hai chiến lược Strict Watermark và Heuristic Watermark được trình bày trong Bảng 10.

Bảng 10: Kết quả thực nghiệm so sánh sự đánh đổi giữa độ trễ chờ và tính hoàn thiện dữ liệu.

Strict Watermark		Heuristic Watermark + DLQ			
$\delta = \text{Wait (s)}$	Completeness (%)	Phân vị p	L_{eff} (s)	Tức thời (%)	Cuối cùng (%)
0	8.75	0.500	11.63	64.21	100.00
1	12.88	0.500	11.63	64.21	100.00
2	17.86	0.500	11.63	64.21	100.00
3	23.29	0.500	11.63	64.21	100.00
5	34.66	0.500	11.63	64.21	100.00
7	45.48	0.500	11.63	64.21	100.00
10	59.17	0.500	11.63	64.21	100.00
15	75.03	0.500	11.63	64.21	100.00
20	84.44	0.750	18.67	81.26	100.00
30	93.23	0.900	28.80	91.31	100.00
40	96.72	0.950	37.78	94.77	100.00
50	98.41	0.990	59.70	97.70	100.00
60	99.27	0.990	59.70	97.70	100.00
90	99.89	0.999	95.03	98.38	100.00
120	99.97	0.999	95.03	98.38	100.00

Ghi chú: Cột **Cuối cùng (%)** luôn đạt 100% nhờ cơ chế đối chiếu đền bù qua Hàng đợi xử lý muộn (DLQ).

Giải thích các cột trong Bảng 10:

- **Wait (s)**: Thời gian chờ (Wait Time) hay biên an toàn δ mà hệ thống Strict Watermark áp đặt sau khi cửa sổ logic kết thúc trước khi chốt kết quả. Đây chính là khoảng trễ nhân tạo để chờ các sự kiện đến muộn.
- **Strict %**: Độ hoàn thiện dữ liệu tức thời (Immediate Completeness) của chế độ Strict Watermark, tương ứng với mức Wait Time ở cột đầu tiên. Thể hiện tỷ lệ phần trăm dữ liệu được xử lý đúng hạn trong cửa sổ trước khi Watermark đóng.
- p : Phân vị mục tiêu (Target Quantile) sử dụng trong chế độ Heuristic Watermark. $p = 0.50$ nghĩa là ước lượng trung vị của phân phối độ trễ; $p = 0.999$ nghĩa là gần như toàn bộ phân phối (99.9%).
- L_{eff} (s): Độ trễ hiệu dụng (Effective Latency) được ước lượng trực tuyến bởi cấu trúc DDSketch tại phân vị p tương ứng. Đây là ngưỡng thời gian mà Heuristic Watermark dùng để chốt cửa sổ thay vì dùng giá trị cố định δ như chế độ Strict.

- **Imm. %:** Độ hoàn thiện dữ liệu tức thời (Immediate Completeness) của chế độ Heuristic Watermark tại phân vị p và L_{eff} tương ứng. Dữ liệu đến sau khi cửa sổ đã chốt sẽ bị định tuyến sang DLQ.
- **Eventual %:** Độ hoàn thiện dữ liệu cuối cùng (Eventual Completeness) của chế độ Heuristic Watermark sau khi đã đối chiếu đèn bù toàn bộ dữ liệu từ Hàng đợi xử lý muộn (DLQ). Giá trị luôn đạt **100%** chứng minh rằng cơ chế DLQ đảm bảo không mất mát dữ liệu.

7.1.2 Các phát hiện và Phân tích chuyên sâu từ thực nghiệm

1. **Mô hình hóa đường cong và Chi phí biên (Marginal Cost Analysis):** Đường cong độ hoàn thiện thực nghiệm của Strict Watermark bám rất sát hàm phân phối tích lũy CDF của độ trễ thực tế ($R^2 \approx 0.876$). Phân tích chi phí biên (Wait Time cần thiết để tăng 1% độ hoàn thiện) chỉ ra quy luật lợi ích giảm dần cực kỳ rõ rệt:
 - Trong khoảng δ từ 0 đến 10 giây, chi phí biên rất thấp, chỉ mất khoảng **176 đến 242 ms** thời gian chờ cho mỗi 1% dữ liệu thu hồi thêm.
 - Khi tiệm cận mức hoàn thiện cao, chi phí biên tăng vọt một cách phi tuyến tính. Để tăng độ hoàn thiện từ 99.0% lên 99.9% (chênh lệch chỉ 0.9%), hệ thống phải tăng δ từ 60s lên 120s, tức chi phí biên lên tới **66,666 ms chờ đợi cho mỗi 1% hoàn thiện**.
 - Để đạt độ hoàn thiện tuyệt đối (tránh mất mát 833 sự kiện ngoại lai có độ trễ $> 120s$), thời gian trễ bắt buộc phải cấu hình $\delta \geq 360$ giây. Đây là mức trễ không thể chấp nhận được đối với các hệ thống phân tích thời gian thực.
2. **Cơ chế thích ứng động của Heuristic Watermark qua DDSketch:** Chế độ Heuristic giải phóng hệ thống khỏi sự phụ thuộc vào cấu hình tĩnh bằng cách sử dụng cấu trúc dữ liệu phác thảo DDSketch ($\alpha = 0.01$) để tính toán động độ trễ hiệu dụng L_{eff} (streaming quantiles) từ dòng dữ liệu thực tế:
 - Quá trình hội tụ trực tuyến của L_{eff} diễn ra rất mượt mà và ổn định. Với phân vị đích $p = 0.90$, giá trị L_{eff} bắt đầu ở 27.77s sau 50,000 sự kiện đầu tiên và hội tụ chính xác về 28.80s ở cuối luồng dữ liệu.
 - Tại $p = 0.50$, hệ thống tự động duy trì thời gian chờ $L_{\text{eff}} \approx 11.63$ giây (nhỏ hơn 10 lần so với mức $\delta = 120s$ của Strict để đạt cùng mức hoàn thiện cuối cùng).
 - Dù độ hoàn thiện tức thời chỉ đạt khoảng 64.21%, cơ chế đối chiếu đèn bù qua Hàng đợi xử lý muộn (DLQ) đảm bảo **tính hoàn thiện cuối cùng đạt tuyệt đối 100%** mà không gây tắc nghẽn hay trì hoãn các cửa sổ xử lý chính.
3. **Ảnh hưởng của kích thước cửa sổ (Window Size Sensitivity Analysis):** Thực nghiệm quét nhảy cảm cho thấy kích thước cửa sổ lớn hơn (ví dụ từ 1s tăng lên 60s) làm giảm số lượng cửa sổ logic và giảm thiểu sai số biên ở các điểm đóng cửa sổ, từ đó tăng độ hoàn thiện tự nhiên khi không chờ ($\delta = 0s$ tăng từ 2.34% lên 77.76%). Tuy nhiên, đối với các trường hợp $\delta \geq 40s$, đường cong độ hoàn thiện hội tụ về cùng một dạng phân phối bất kể kích thước cửa sổ, chứng tỏ phân phối độ trễ dòng sự kiện mới là yếu tố quyết định hiệu năng.

7.1.3 Khuyến nghị cấu hình vận hành thực tế (Production Guidelines)

Dựa trên kết quả thực nghiệm, chúng tôi đề xuất các mô hình cấu hình tối ưu tùy thuộc vào mục tiêu nghiệp vụ của hệ thống:

- **Dashboard trực quan thời gian thực (Real-time Dashboards):** Sử dụng chế độ Strict Watermark với $\delta = 38s$. Mô hình này cung cấp độ hoàn thiện tức thời $\approx 95\%$ với độ trễ xử lý cố định và dễ dự đoán, loại bỏ hoàn toàn chi phí xử lý đền bù phức tạp của DLQ.
- **Hệ thống phân tích và đối chiếu tài chính (Analytics/Financial Ledgers):** Sử dụng chế độ Heuristic Watermark với phân vị $p = 0.50$ ($L_{\text{eff}} \approx 12s$). Cấu hình này giúp đóng cửa sổ nhanh nhất, giải phóng trạng thái trong RocksDB ngay lập tức, trong khi DLQ đảm bảo tính chính xác và đầy đủ tuyệt đối 100% của dữ liệu.
- **Chế độ vận hành cân bằng (Balanced Operations):** Sử dụng chế độ Heuristic Watermark với phân vị $p = 0.90$ ($L_{\text{eff}} \approx 29s$). Lựa chọn này đem lại độ hoàn thiện tức thời cao (91.31%), do đó lượng dữ liệu trễ phải đi qua DLQ chỉ chiếm dưới 10%, giảm thiểu tối đa tài nguyên tính toán cho pha đền bù dữ liệu.

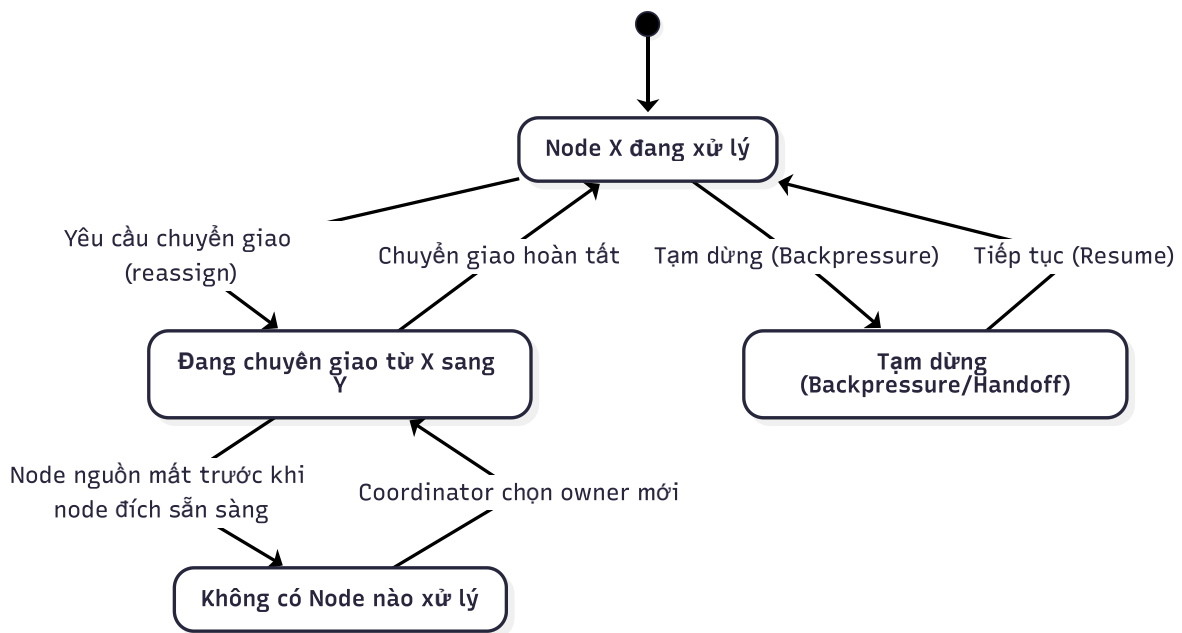
7.2 Failure Scenarios & Recovery Path (Kịch bản sự cố & Quy trình phục hồi)

1. Sập Worker Node (Worker Node Crash):

- **Sự cố (Failure Scenario):** Khi một Worker (ví dụ: Worker 3 phụ trách các phân mảnh $P_9 - P_{11}$) bị dừng tiến trình đột ngột:
 - **Phát hiện lỗi:** Coordinator không nhận được nhịp tim từ Worker 3 quá ngưỡng 10 giây, lập tức xác nhận Worker 3 đã sập.
 - **Phong tỏa và Đối nhiệm kỳ (Fencing):** Coordinator tăng nhiệm kỳ (term) của Fencing Token để vô hiệu hóa tất cả các lệnh cũ từ Worker 3 nếu nó sống dậy bất ngờ.
 - **Tái phân bổ & Cân bằng tải (Rebalance):** Coordinator chuyển các phân mảnh P_9, P_{10}, P_{11} sang trạng thái đang tái phân bổ (REASSIGNING) trong nhật ký đồng thuận. Để đảm bảo cân bằng tải và tránh hiện tượng nút cổ chai, Coordinator không chỉ định một node duy nhất gánh toàn bộ, mà chia đều các phân mảnh bị ảnh hưởng cho các Worker còn sống:
 - Phân mảnh P_9 được gánh hộ bởi Worker 0 (node0).
 - Phân mảnh P_{10} được gánh hộ bởi Worker 1 (node1).
 - Phân mảnh P_{11} được gánh hộ bởi Worker 2 (node2).
 - **Khôi phục trạng thái ấm (Warm State Recovery):** Các Worker còn sống (Worker 0, Worker 1, Worker 2) khi nhận chỉ thị tiếp quản phân mảnh tương ứng sẽ nạp checkpoint từ Shared Volume (Tier-2) bao gồm offset đã xử lý gần nhất và siêu dữ liệu trạng thái, tiếp tục tiêu thụ từ offset kế tiếp trên Kafka để xử lý dòng sự kiện, bảo đảm tính nhất quán Exactly-Once.
- **Phục hồi (Recovery Path - Failback):** Khi Worker 3 được khởi động lại và sẵn sàng nhận lại các phân mảnh cũ của mình (P_9, P_{10}, P_{11}):

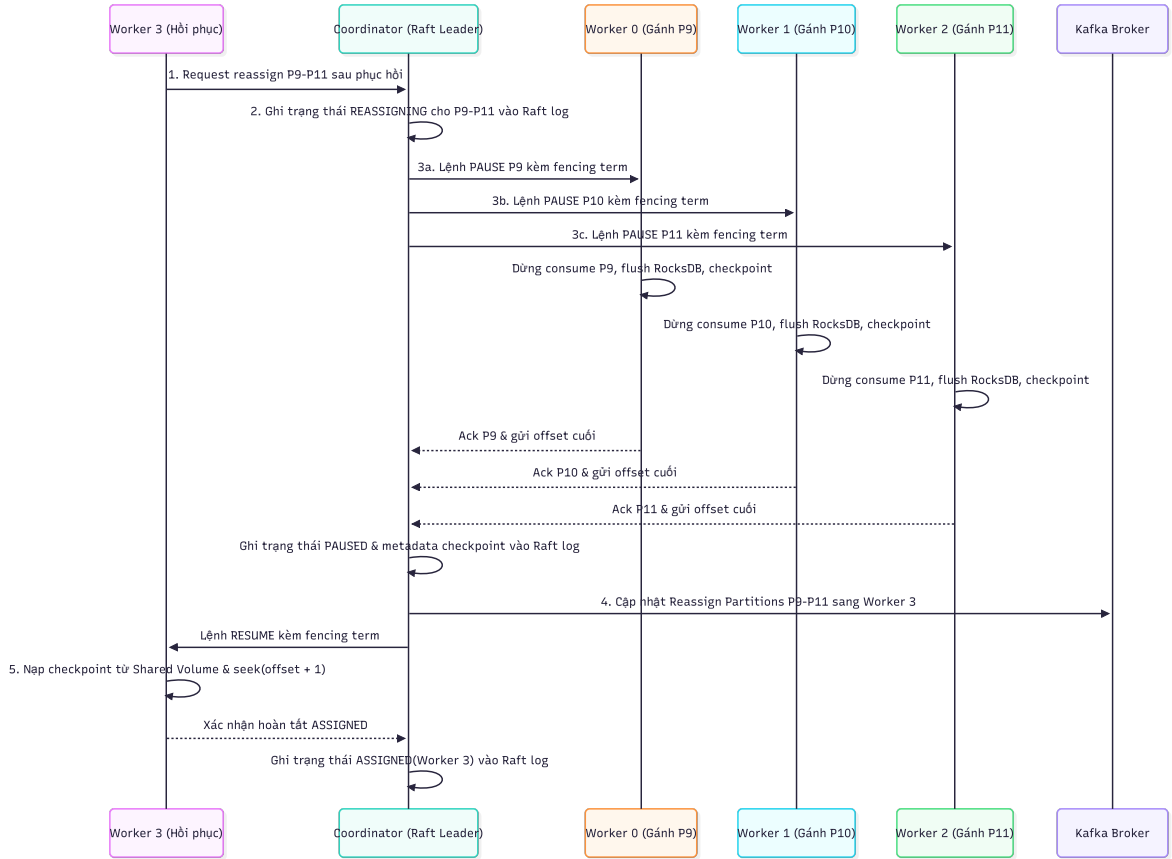
- **Bước 1 (Request):** Worker 3 gửi yêu cầu nhận lại các phân mảnh cũ lên Coordinator.
- **Bước 2 (Lock):** Coordinator ghi nhận trạng thái đang tái phân bổ (REASSIGNING) cho các phân mảnh P_9, P_{10}, P_{11} vào nhật ký đồng thuận.
- **Bước 3 (Pause Source):** Coordinator gửi lệnh tạm dừng (PAUSE) đồng thời đến các Worker đang gánh hộ tạm thời (Worker 0, Worker 1, Worker 2). Các Worker này dừng tiêu thụ phân mảnh bàn giao tương ứng, flush toàn bộ trạng thái của sổ xuống đĩa, ghi checkpoint hoàn chỉnh lên Shared Volume và gửi xác nhận kèm offset cuối cùng đã xử lý về Coordinator.
- **Bước 4 (Reassign):** Coordinator chuyển quyền sở hữu các phân mảnh $P_9 - P_{11}$ cho Worker 3, thu hồi quyền xử lý từ các Worker đang gánh hộ, ghi nhận trạng thái đã tạm dừng (PAUSED) vào nhật ký đồng thuận.
- **Bước 5 (Resume Target):** Coordinator gửi lệnh tiếp tục (RESUME) kèm Fencing Token cho Worker 3. Worker 3 nạp checkpoint từ Shared Volume, tiếp tục tiêu thụ từ offset kế tiếp trên Kafka cho cả 3 phân mảnh và bắt đầu xử lý bình thường. Trạng thái các phân mảnh trở về đã gán (ASSIGNED) trong nhật ký đồng thuận.

Sơ đồ máy trạng thái phân mảnh (Partition State Machine):



Hình 8: Sơ đồ máy trạng thái phân mảnh (Partition State Machine).

Sơ đồ trình tự Failback 5 bước sau phục hồi:



Hình 9: Sơ đồ trình tự Failback 5 bước sau phục hồi.

2. *Phân mảnh mạng (Split-brain Fencing)*: Sử dụng Fencing Token gắn nhiệm kỳ (term) đơn điệu tăng trên các chỉ thị điều phối để ngăn chặn các Coordinator cũ gửi lệnh sai lệch tới Worker khi mạng bị chia cắt.
3. *Hiện tượng Idle Partition*: Giả lập một phân mảnh không có log phát sinh. Worker tự động đánh dấu phân mảnh này là nhân rồi tường minh (explicit idle) khi vượt ngưỡng thời gian, để Coordinator tạm thời loại trừ khỏi hàm min tính W_{global} , ngăn chặn hiện tượng nghẽn dòng chảy thời gian của cả hệ thống.